

FAULT ATTACKS AGAINST UOV-BASED SIGNATURES

SVEN BAUER[✉], FABRIZIO DE SANTIS[✉], KRISTJANE KOLECI[✉]

Siemens AG, Foundational Technologies, Munich, Germany

ABSTRACT. The Unbalanced Oil and Vinegar (UOV) construction is the foundation of several post-quantum digital signature algorithms currently under consideration in NIST’s standardization process for additional post-quantum digital signature schemes. This paper introduces new single fault injection attacks against the signing procedure of deterministic variants of signature schemes based on the UOV construction. We show how these attacks can be applied to attack MAYO and PROV, two signature schemes submitted to the NIST call for additional post-quantum signature schemes. The attacks are demonstrated with reference implementations that run on an ARM Cortex-M4 processor. Our attacks do not require precise triggering or precise fault injection capabilities. Any type of fault in large portions of the code has the potential to result in successful key recovery. We demonstrate our attacks with very cheap equipment and simple clock glitching techniques, enabling the recovery of the secret key with either two faulty signatures or one correct signature and one faulty signature in the case of MAYO and one correct signature and two faulty signatures in case of PROV. The fact that our attacks do not require precise fault injection capabilities and can be successful with only a few signatures makes them particularly powerful, hence harmful for the implementation security of post-quantum digital signature schemes.

1. INTRODUCTION

Post-quantum cryptography is becoming more and more established and recently standards for the post-quantum signature schemes ML-DSA and SLH-DSA have been published by NIST. In 2022 NIST started a new standardization process for more post-quantum signature schemes to have a broader variety of schemes based on different mathematical principles. Signatures based on the UOV construction are interesting because they offer very compact signatures. Four of these schemes (MAYO, QR-UOV, SNOVA and UOV) have been promoted to the second round of the NIST process. This paper and other recent works [14, 27, 20, 4, 18, 2, 3] show that the implementation security of these schemes still needs to be investigated.

The present work describes the first fault attacks that exploit determinism in signature schemes in the UOV family and the first attack that is based on changing the ‘oil’ component of signatures.

1.1. Structure. The paper is structured as follows: Section 3 provides background information on the oil and vinegar schemes. In Section 4 we describe our attacks in the general UOV setting. Afterwards, we investigate these attacks in the concrete settings of deterministic MAYO in Section 5 and PROV in Section 6, respectively. In Section 7 we describe practical implementations of some of our fault attacks

E-mail address: {svenbauer, fabrizio.desantis}@siemens.com.

Key words and phrases. Multivariate Cryptography; Post-Quantum Cryptography; UOV; Mayo; PROV; Fault Attacks.

against MAYO and PROV on an embedded board. Finally, we discuss possible countermeasures and give an outlook on possible extensions of this work.

2. NOTATION

Our notation largely follows [2]. We use bold lower-case letters like $\mathbf{x}$ for (column) vectors and bold upper-case letters like $\mathbf{A}$ for matrices. The finite field with q elements is denoted as $\mathbb{F}_q$.

3. BACKGROUND

This paper is about fault attacks against the signing procedure of deterministic unbalanced oil and vinegar (UOV) signature schemes. To fix notation and to make this paper self-contained we therefore first explain how signing works in these schemes. For more detailed explanations of UOV signature schemes, the key generation and the verification procedure, we refer the reader to the extensive existing literature on the topic. UOV signature schemes in general and MAYO in particular are described in [8, 11]. A good overview of fault attacks against UOV-based signature schemes can be found in [2]. Analytical tools for attacks are explained in [8].

3.1. UOV. We first describe UOV signature schemes in general. The main system parameters of a UOV signature scheme are a finite field $\mathbb{F}_q$, positive integers n, m, o with $n > 2m$ and $n > o$. We also need a hash function that maps byte strings to vectors in $\mathbb{F}_q^n$.

The public key of a UOV signature scheme is a map

$$(1) \quad \mathcal{P} : \mathbb{F}_q^n \rightarrow \mathbb{F}_q^m$$

that consists of m homogenous quadratic polynomials.

The corresponding private key is a basis $\mathbf{o}_1, \mathbf{o}_2, \dots, \mathbf{o}_o \in \mathbb{F}_q^n$ of a vector subspace $O \subset \mathbb{F}_q^n$ with $\dim O = o$ such that $\mathcal{P}$ vanishes on O .

To describe the signing procedure we need the fact that the *polar form* of $\mathcal{P}$ defined as

$$(2) \quad \begin{aligned} \mathcal{P}' : \mathbb{F}_q^n \times \mathbb{F}_q^n &\rightarrow \mathbb{F}_q^m, \\ \mathcal{P}'(\mathbf{x}, \mathbf{y}) &:= \mathcal{P}(\mathbf{x} + \mathbf{y}) - \mathcal{P}(\mathbf{x}) - \mathcal{P}(\mathbf{y}) \end{aligned}$$

is a symmetric bilinear map (Theorem 1 in [8]).

To sign a message M , the signer first computes a hash of the message

$$(3) \quad \mathbf{t} := h(M) \in \mathbb{F}_q^m.$$

A valid signature for M is a vector $\mathbf{s} \in \mathbb{F}_q^n$ such that

$$(4) \quad \mathcal{P}(\mathbf{s}) = \mathbf{t}$$

To find such a vector $\mathbf{s}$, the signer picks a (pseudo-)random $\mathbf{v} \in \mathbb{F}_q^n$ (the ‘vinegar vector’), and then solves the linear system of equations

$$(5) \quad \mathbf{t} - \mathcal{P}(\mathbf{v}) = \mathcal{P}'(\mathbf{v}, \mathbf{o})$$

for $\mathbf{o} \in O$ (the ‘oil vector’) and sets $\mathbf{s} := \mathbf{v} + \mathbf{o}$. This system of equations for $\mathbf{o}$ is linear, because $\mathcal{P}'$ is a bilinear map. The system parameters have to be chosen in such a way that such a solution exists with high probability. If solving the system fails, the signer picks a different $\mathbf{v}$ and tries again.

The attacks described in this paper apply primarily to deterministic oil and vinegar signature schemes. Hence, we assume that $\mathbf{v}$ is pseudo-randomly generated with a pseudo random number generator seeded by the message M and a secret seed value that is part of the secret key.

The vector $\mathbf{s}$ satisfies (4) because

$$(6) \quad \begin{aligned} \mathcal{P}(\mathbf{s}) &= \mathcal{P}(\mathbf{v} + \mathbf{o}) \stackrel{(2)}{=} \mathcal{P}'(\mathbf{v}, \mathbf{o}) + \mathcal{P}(\mathbf{v}) + \mathcal{P}(\mathbf{o}) \\ &\stackrel{(5)}{=} \mathbf{t} + \mathcal{P}(\mathbf{o}) = \mathbf{t}. \end{aligned}$$

We have used the fact that $\mathbf{o} \in O$ and that $\mathcal{P}$ vanishes on O by definition of O .

An algorithmic description of the signing procedure is given in Algorithm 1.

Algorithm 1 Unbalanced oil and vinegar signature generation

Require: A secret key consisting of a basis $\mathbf{O} = (\mathbf{o}_1, \mathbf{o}_2, \dots, \mathbf{o}_o)$ of O , a secret *seed* as a string of bytes, the public key $\mathcal{P}$, a message M

Ensure: $\mathbf{s} \in \mathbb{F}_q^n$ is a valid signature, i.e., satisfies $\mathcal{P}(\mathbf{s}) = h(M)$

```

1:  $\mathbf{t} \leftarrow h(M)$ 
2: PRNG_init( $M \parallel \text{seed}$ )
3: loop
4:    $\mathbf{v} \leftarrow \text{PRNG\_generate}()$ 
5:    $\mathbf{y} \leftarrow \mathbf{t} - \mathcal{P}(\mathbf{v})$ 
6:   Build a matrix  $\mathbf{A}$  of the linear map  $O \rightarrow \mathbb{F}_q^m$ ,  $\mathbf{x} \mapsto \mathcal{P}'(\mathbf{v}, \mathbf{x})$  with respect to
     the basis  $(\mathbf{o}_i)$  of  $O$  and the canonical basis of  $\mathbb{F}_q^m$ .
7:    $r \leftarrow \text{PRNG\_generate}()$ 
8:   Solve  $\mathbf{A}\mathbf{x} = \mathbf{y}$  for  $\mathbf{x}$ . Pick a solution from the solution space pseudo-
     randomly based on  $r$ .
9:   if  $x \neq \perp$  then
10:    break
11:  $\mathbf{s} \leftarrow \mathbf{v} + \mathbf{O}\mathbf{x}$ 
12: return  $\mathbf{s}$ 

```

3.2. Related work. In most existing works on fault attacks against signature schemes based on UOV the ‘vinegar variables’ are attacked. The work [27] describes attacks against UOV and Rainbow based on setting some ‘vinegar variables’ to known values.

In [4] two different fault attack strategies against the UOV-based signature scheme Rainbow are presented. As in [27], the first attack is based on setting the ‘vinegar variables’ to a fixed value. Concretely, [4] achieves this by skipping their initialization. The paper then shows how this leads to full key recovery. A second attack is based on skipping parts of the computation.

The attack in [18] is essentially a translation of the ideas of [27] to the UOV-based signature scheme MAYO. It is also based on forcing the ‘vinegar variables’ to a fixed value.

In [3] the authors describe another attack based on injecting a fault into the initialization of the ‘vinegar variables’. The attack is described for MAYO. The authors also explain how ideas from the (purely cryptanalytic) reconciliation attack can be used to achieve key recovery with just one correct and one faulty signature. The reconciliation attack as described, for example, in [9, 24] is a method for completing a basis of the secret oil space O given only a single non-zero vector in O .

The work [14] describes a fault attack based on changing coefficients of the central map.

The paper [2] gives an excellent overview of the current state of research into side-channel and fault attacks against UOV-based signature schemes, and we refer the interested reader to it for further background information. In addition, it also briefly describes a new attack based on injecting a fault into the preparation of the

matrix in the system of linear equations that needs to be solved for generating a signature.

3.3. Contributions. To the best of our knowledge, this is the first paper that presents attacks based on injecting a fault that leads to different values in the ‘oil variables’. Our attacks are applicable to deterministic variants of UOV-based signatures scheme. We show how our attack can concretely be applied to attack the signing procedures of the signature schemes MAYO and PROV. In all cases, we achieve key recovery with one correct and one faulty signature for the same message. The attacks have been simulated, and the paper presents experimental verification of two of our fault attacks against deterministic MAYO and PROV. Our attacks work with a single fault injected in a large time window and with a very liberal fault model (skipping of one or several instructions as well as changes in data both lead to exploitable faults). Hence, it seems plausible that our attacks are easier to implement in practice than most of the attacks in the existing literature.

4. OUR FAULT ATTACKS

First, we give an abstract description of our fault attacks in the general UOV framework. Then we will explain how to specialize the attack to MAYO and PROV.

It is known that fixing $\mathbf{v}$ allows an attack [27, 4]. All known attacks, which we have discussed in the previous section, achieve this by skipping the initialization of $\mathbf{v}$. This means injecting a fault in Line 4 of Algorithm 1.

The new idea here is to achieve the same effect in deterministic variants of unbalanced oil and vinegar schemes by re-signing the same message and injecting a fault to force a different target vector or a different solution for the ‘oil’ variables. We will describe two fault attack scenarios to achieve this.

4.1. Attack 1: Faulting the target vector. In the first scenario the attacker injects a fault in Line 1 of Algorithm 1. A single fault leads to a different value $\mathbf{t}' \neq \mathbf{t}$. This in turn leads to a different value $\mathbf{y}' = \mathbf{t}' - \mathcal{P}(\mathbf{v})$ and hence a different value $\mathbf{x}' \neq \mathbf{x}$ and finally a different signature $\mathbf{s}' = \mathbf{v} + \mathbf{O}\mathbf{x}'$. Hence, with a correct and a faulty signature for the same message M , the attacker can compute

$$(7) \quad \mathbf{s} - \mathbf{s}' = \mathbf{O}(\mathbf{x} - \mathbf{x}').$$

This is a non-zero vector in the secret linear subspace O .

The attacker can repeat this procedure to find more vectors of O until a basis of O is recovered. However, there is an easier way. This is essentially the reconciliation attack described in, for example, Sec. 4.1 of [9] or [24], which we will explain in the present context. Let us assume the attacker has recovered j linearly independent vectors $\mathbf{o}_1, \mathbf{o}_2, \dots, \mathbf{o}_j \in O$. The attacker now considers the linear map:

$$(8) \quad \begin{aligned} f_j : \mathbb{F}_q^n &\rightarrow \mathbb{F}_q^{jm} \\ f_j(\mathbf{x}) &= (\mathcal{P}'(\mathbf{o}_1, \mathbf{x}), \mathcal{P}'(\mathbf{o}_2, \mathbf{x}), \dots, \mathcal{P}'(\mathbf{o}_j, \mathbf{x})). \end{aligned}$$

By standard linear algebra we have:

$$(9) \quad jm \geq \dim \operatorname{im}(f_j) = n - \dim \ker(f_j).$$

Hence,

$$(10) \quad \dim \ker(f_j) \geq n - jm.$$

Because $O \subset \ker(f_j)$, we have

$$(11) \quad \dim O \leq \dim \ker(f_j).$$

We also see that:

$$(12) \quad \ker(f_{j+1}) \subset \ker(f_j).$$

TABLE 1. Overview of attacks in related works in relation to the attacks described in this paper.

Attack	Source	Target
Fix vinegar variables	[27, 4, 18, 3]	Algorithm 1, Line 4
Partial computation skip	[4]	Algorithm 1, Line 6
Fault coefficient in central map	[14]	Algorithm 1, Line 6
Fault target vector in deterministic variant	this work	Algorithm 1, Line 1
Fault solution sampling in deterministic variant	this work	Algorithm 1, Line 8

The attacker injects j faults such that $n - jm \leq o$. Because of Eq. (12) the kernel $\ker(f_j)$ can become small enough such that $O = \ker(f_j)$. So a basis of this kernel is a basis of O and therefore equivalent to the original secret basis of O . Signatures created with this equivalent private key are indistinguishable from signatures created with the original key. We will see in the next section that for all MAYO variants a single fault (i.e., $j = 1$) is sufficient for full key recovery.

Note that any fault that changes the output of h in Line 1 of Algorithm 1 works. Because hash functions take some time to compute, this means that the attacker has a large time window to inject a fault. There are also many faults which can change the output of a hash function. Instruction skips or data faults would both work, and it does not matter if the fault affects more than one instruction. So, the fault model is very liberal, and it should generally be easy to inject a suitable fault, if it is possible to inject any fault at all.

4.2. Attack 2: Faulting the solution sampling. For our second attack scenario, the attacker injects a fault in Line 7 of Algorithm 1. This will lead to a different value $r' \neq r$ and hence to a different solution $\mathbf{x}' \neq \mathbf{x}$ being picked and again to a different signature $\mathbf{s}' = \mathbf{v} + \mathbf{O}\mathbf{x}'$. For recovering a basis of the secret space O , the attacker proceeds as in our previous attack.

This time, the fault has to be injected during the output generation in a pseudo-random number generator. As in the previous attack, the attacker has a large time window to inject the fault and a variety of different faults has a suitable effect. So, this is again a fault attack that is expected to be relatively easy to perform in practice. In particular, we expect this to be easier in practice than setting variables to particular values as required by the attacks in [27].

An interesting property of the attacks in 4.1 and 4.2 is that they can also be performed without a correct signature, because Eq. (7) also holds if both $\mathbf{s}$ and $\mathbf{s}'$ are faulty signatures. In practice, however, correct signatures are easier to obtain than faulty ones, so under normal circumstances an attacker will use one correct signature and as many faulty ones as needed.

Table 1 shows an overview of the attacks described in Section 3.2 in relation to the attacks described in this paper.

5. ATTACKING DETERMINISTIC MAYO

5.1. The MAYO signature scheme. MAYO [10] is a signature scheme based on the unbalanced oil and vinegar construction. MAYO has been submitted to NIST for the standardization of additional post-quantum signature schemes. In October 2024 NIST promoted MAYO to the second round of this standardization

process. The specification of MAYO [10] proposes four different variants of MAYO – called MAYO₁, MAYO₂, MAYO₃ and MAYO₅ – for standardization. The variants differ in the dimensions n , m and o of the vector spaces defining the scheme. In addition to classic UOV as described in Section 3.1, MAYO has a so-called ‘whipping-parameter’ k . This is related to a MAYO-specific technique to reduce the size of the oil space and the public key. This ‘whipping up’ technique is not of importance in the present context, and we therefore refer the interested reader to the MAYO specification [10] for details. The exact values of the parameters n , m , o and k for the four MAYO variants will be important later and are listed in Table 2.

The innovative technique for reducing the size of the oil space and hence the public key is the main difference between MAYO and classic unbalanced oil and vinegar schemes as described in Section 3.1. We do not describe the full MAYO signature scheme here, because this is not required to make this paper self-contained. We will only describe the procedure of generating a signature and refer the reader to the MAYO specification [10] for details. The general procedure for creating a signature in any unbalanced oil and vinegar scheme. For MAYO this procedure is shown in Algorithm 2. The signing procedure of MAYO contains an optional randomization. Our attack is applicable to deterministic signature schemes, so we only consider the deterministic version of MAYO. In this case, R is set to a string of 0-bytes in Line 3 of Algorithm 2.

Algorithm 2 MAYO signature generation

Require: A MAYO secret key esk , a message M

Ensure: sig is a valid MAYO signature of M under esk

```

1:  $\text{seed}, \mathbf{O}, (\mathbf{P}_i^{(1)})_{i \in m}, (\mathbf{L}_i)_{i \in m} \leftarrow \text{Decode}(\text{esk})$ 
2:  $M\_digest \leftarrow \text{SHAKE256}(M)$ 
3:  $R \leftarrow \text{salt\_bytes}$  0-bytes or  $R \leftarrow \text{salt\_bytes}$  random bytes  $\triangleright$  Optional randomization
4:  $\text{salt} \leftarrow \text{SHAKE256}(M\_digest \| R \| \text{seed})$ 
5:  $\mathbf{t} \leftarrow \text{SHAKE256}(M\_digest \| \text{salt})$ 
6: for  $\text{ctr} = 0, \dots, 255$  do
7:    $\mathbf{v}_0, \dots, \mathbf{v}_{k-1}, r \leftarrow \text{SHAKE256}(M\_digest \| \text{salt} \| \text{seed} \| \text{ctr})$ 
8:    $(\mathbf{A}, \mathbf{y}) \leftarrow$ 
        $\text{BuildLinearEquations}((\mathbf{v}_i)_{i \in k}, \mathbf{P}^{(1)}, (\mathbf{L}_i)_{i \in m}, \mathbf{t})$ 
9:    $\mathbf{x} \leftarrow \text{SampleSolution}(\mathbf{A}, \mathbf{y}, r)$ 
10:  if  $x \neq \perp$  then
11:    break
12:  $\mathbf{s} \leftarrow (\mathbf{v}_i + \mathbf{O}\mathbf{x}_i \| \mathbf{x}_i)_{i \in k}$ 
13:  $\text{sig} \leftarrow \mathbf{s} \| \text{salt}$ 
14: return  $\text{sig}$ 
```

5.2. Fault injection and key recovery with attack 1. We translate attack from Section 4.1 to MAYO. The attacker starts by interacting with the device under attack as follows:

- (1) Sign an arbitrary message M on the device under attack. The result is a signature vector $\mathbf{s}$.
- (2) Sign the same message M again, but this time inject a fault in Algorithm 2, Line 5 such that a different vector $\mathbf{t}' \neq \mathbf{t}$ is generated. The result is a faulty signature vector $\mathbf{s}'$.

TABLE 2. The values of n , m , o and k for the proposed standard variants of MAYO (NIST round 1) in [10]. The value k is the so-called ‘whipping parameter’, which is not present in other UOV-based signature schemes.

	MAYO ₁	MAYO ₂	MAYO ₃	MAYO ₅
n	66	78	99	133
m	64	64	96	128
o	8	18	10	12
k	9	4	11	12

TABLE 3. As Table 2, but with the revised values in NIST round 2 in [11].

	MAYO ₁	MAYO ₂	MAYO ₃	MAYO ₅
n	86	81	118	154
m	78	64	108	142
o	8	17	10	12
k	10	4	11	12

As was already pointed out in Section 4, it is important to note that the type of fault required for this attack is one that is relatively easy to achieve. All that is required is that the call to `SHAKE256` in Line 5 returns an incorrect result. Considering the structure of `SHAKE256` [13], we see that this can be achieved by an instruction skip or any other fault during the absorption phase of `SHAKE256` or a fault during the computation of a Keccak round at the core of `SHAKE256`. So the fault model is very liberal (instruction skip or data fault) and the time window during which a fault can be injected is relatively large.

The fault does not change the $\mathbf{v}_i$ or r generated in Algorithm 2, Line 5. Because the target vector $\mathbf{t}'$ is different, `BuildLinearEquations` in Line 8 will output a target vector $\mathbf{y}' \neq \mathbf{y}$. Therefore, Line 9 will find a different solution $\mathbf{x}' \neq \mathbf{x}$. Therefore,

$$(13) \quad \mathbf{o}_1 := \mathbf{s}' - \mathbf{s} = (\mathbf{O}(\mathbf{x}'_i - \mathbf{x}_i))_{i \in k} \in \mathcal{O}.$$

So, with one correct and one faulty signature the attacker obtains a non-zero oil space vector $\mathbf{o}_1$.

From Table 2 and Table 3 we see that for all proposed standard variants of MAYO in round 1 and round 2 of the NIST process we have $n - m \leq o$, so a single non-zero oil space vector $\mathbf{o}_1$, and hence a single fault, is sufficient for full key recovery, as we have seen in Section 4.

5.3. Fault injection and key recovery with attack 2. We translate the attack from Section 4.2 to MAYO. To mount the attack, the attacker proceeds as follows:

- (1) Sign an arbitrary message M on the device under attack. The result is a signature vector $\mathbf{s}$.
- (2) Sign the same message M again, but this time inject a fault in Line 7. This line generates pseudo-random bytes to initialize first the $\mathbf{v}_i$ and then r . The fault must be injected towards the end of the `SHAKE256` procedure so that the fault only affects the value r but not the values $\mathbf{v}_i$. This results in a value $r' \neq r$ and then in the sampling of a different solution $\mathbf{x}' \neq \mathbf{x}$ in Line 9.

Because the variables $\mathbf{v}_i$ are not affected by the fault, the attacker is again in the situation of (13) and can use the same techniques as in Section 4.1 to recover more

oil vectors and hence an (equivalent) secret key. As in our attack in Section 5.2, one correct and one faulty signature are sufficient for full key recovery.

We will now look at the time window the attacker has for injecting a suitable fault. To generate the $\mathbf{v}_i$ the call to SHAKE256 has to output $k \lceil (n - o)/2 \rceil$ bytes. To generate r another $ko/2$ bytes have to be produced (see [10], Algorithm 8, Line 19). The sponge construction in SHAKE256 ‘squeezes out’ 136 bytes at a time and then calls the Keccak round function.

Hence, for MAYO₁ 261 bytes are required for the $\mathbf{v}_i$ and r is 36 bytes in size. Two calls to the Keccak round function produce $2 \cdot 136 = 272$ bytes. This leaves 16 bytes over for r . Because r is 36 bytes in size, 20 bytes more and hence another call to the Keccak round function are required. A fault in the Keccak round function, typically resulting in one or more skipped instructions, is very likely to change the output of the Keccak round function and hence the value of r . Therefore, the attacker has again a large time window to inject a fault and the effect of the fault does not have to target a particular instruction precisely. Any fault that changes the output of the Keccak round function is sufficient to mount the attack.

Similar considerations show that also in MAYO₂ and MAYO₃ an extra Keccak round function call is required when generating r . Hence, the attacker also has a large time window for injecting a suitable fault when attacking MAYO₂.

However, for MAYO₅ no extra Keccak round function call is required to generate r . Hence, in these cases the attack is a bit more difficult, because the attacker would have to target, for example, the copying of the output from the Keccak state buffer to the memory location of r . Nevertheless, because r is $12 \cdot 12/2 = 72$ bytes in size, the copy loop is still large enough to be a relatively easy target for a fault that leads to the skipping of one or more instructions.

6. ATTACKING PROV

6.1. The PROV signature scheme. PROV [16] is another signature scheme that is based on the UOV construction and that was submitted to NIST for standardization as an additional post-quantum signature scheme. In October 2024 NIST decided not to promote PROV to the second round of the standardization process. Nevertheless, we believe looking at PROV is worthwhile, because it gives another example how our attacks described in the general framework of UOV in Section 4 can be instantiated for a concrete signature scheme.

As with MAYO, we only give a description of the signing procedure here, because details of key generation and signature verification in PROV are not relevant in the present context. We refer the interested reader to [16] for a full specification of the PROV signature scheme. The PROV signature generation algorithm is given in Algorithm 3. We leave out some details like the expansion of $\mathbf{O}$ because they are not relevant in the present context.

The PROV specification [16] defines three different variants – called PROV-I, PROV-III and PROV-V – addressing three different security levels. The three variants differ in the dimensions of the vector spaces that define the scheme. The values are given in Table 4. The variables n , m and o have exactly the same meaning as in our general description of UOV schemes in Section 3.1.

PROV uses a different strategy from MAYO to determine a single solution of the system of linear equations during the signing procedure if there are several solutions (which generally is the case). The procedure `LinSolve` that is called in Line 14 of Algorithm 3 is defined in the PROV specification [16] in such a way that it deterministically returns a particular solution, independently of other details of the linear solver. This means that our attack in Section 4.2 (fault injection to pick

Algorithm 3 PROV signature generation

Require: A PROV secret key consisting of a public byte string hpk and a secret byte string s_{sk} , a message M

Ensure: sig is a valid PROV signature of M under the given secret key

```

1:  $\text{s}_{\text{pk}} \leftarrow \text{SHAKE256}(0 \parallel \text{s}_{\text{sk}})$ 
2:  $\mathbf{O} \leftarrow \text{Expand}_{\mathbf{O}}(\text{s}_{\text{sk}})$ 
3:  $(\mathbf{P}_i^{(1)}, \mathbf{P}_i^{(2)}) \leftarrow \text{Expand}_{\text{pk}}(\text{s}_{\text{pk}})$ 
4:  $H \leftarrow \text{SHAKE256}.\text{absorb}(4 \parallel \text{s}_{\text{sk}} \parallel M)$ 
5:  $\mathbf{v} \leftarrow H.\text{squeeze}()$ 
6: loop
7:    $\text{salt} \leftarrow H.\text{squeeze}()$ 
8:    $(h_1, \dots, h_m) \in \mathbb{F}_q^m \leftarrow \text{SHAKE256}(\text{hpk} \parallel M \parallel \text{salt})$ 
9:   for  $i = 1, \dots, m$  do
10:     $t_i \leftarrow h_i - \mathbf{v}^T \mathbf{P}_i^{(1)} \mathbf{v}$ 
11:     $\mathbf{a}_i \leftarrow \mathbf{v}^T ((\mathbf{P}_i^{(1)} + \mathbf{P}_i^{(1)^T}) \mathbf{O} + \mathbf{P}_i^{(2)})$ 
12:     $\mathbf{A} \leftarrow (\mathbf{a}_i)_{i=1, \dots, m}$ 
13:     $\mathbf{t} \leftarrow (t_i)_{i=1, \dots, m}$ 
14:     $\mathbf{o} \leftarrow \text{LinSolve}(\mathbf{A}, \mathbf{t})$ 
15:    if  $\mathbf{o} \neq \perp$  then
16:      break
17:  $\mathbf{s} \leftarrow \begin{pmatrix} \mathbf{v} \\ 0 \end{pmatrix} + \begin{pmatrix} \mathbf{O} \\ \mathbf{I}_o \end{pmatrix} \mathbf{o}$ 
18:  $\text{sig} \leftarrow (\mathbf{s}, \text{salt})$ 
19: return  $\text{sig}$ 
    
```

a different solution) is not applicable to PROV. However, our attack in Section 4.1 still works.

6.2. Fault injection and key recovery. To apply our attack from Section 4.1 the attacker injects a fault in one of the SHAKE256 computations in Line 7 or Line 8 in Algorithm 3. The attacker's goal is that a different $\mathbf{h}' \neq \mathbf{h}$ is generated in Line 8. Again, there is a large time window and the precise nature of the fault is not important. Hence, injecting a suitable fault is relatively easy.

A different $\mathbf{h}' \neq \mathbf{h}$ in Line 8 means that a different $\mathbf{t}' \neq \mathbf{t}$ is calculated in Line 10 and Line 13. This in turn leads to a different solution $\mathbf{o}' \neq \mathbf{o}$ in Line 14 and finally a different

$$(14) \quad \mathbf{s}' = \begin{pmatrix} \mathbf{v} \\ 0 \end{pmatrix} + \begin{pmatrix} \mathbf{O} \\ \mathbf{I}_o \end{pmatrix} \mathbf{o}'$$

in Line 19. So the attacker can compute

$$(15) \quad \mathbf{o}_1 := \mathbf{s}' - \mathbf{s} = \begin{pmatrix} \mathbf{O} \\ \mathbf{I}_o \end{pmatrix} (\mathbf{o}' - \mathbf{o})$$

and again obtains a vector $\mathbf{o}_1 \in O$.

From Table 4 we see that for PROV-I $o > n - m$, but $o < n - 2m$. Hence, two faults are sufficient for key recovery according to Section 4. Similarly, two faults are also required to break PROV-III and PROV-V.

7. EXPERIMENTS

To verify the attack in Section 4.1, we first worked with simulated faults. We wrote a script **attack** to recover the secret oil space from a correct and a faulty signature we provided and a second script **forg** that, given the secret oil space

TABLE 4. The values of n , m and o for the proposed standard variants of PROV in [16].

	PROV-I	PROV-III	PROV-V
n	142	206	270
m	49	74	100
o	57	82	108

output by **attack**, produces a signature for a given input. We verified that running **attack** on a pair of signatures, one correct and one with a simulated fault, produced an oil space and that running **forge** with the recovered oil space produced a valid signature.

In a second step, we used a ChipWhisperer board to generate a faulty signature s' . The ‘real’ faulty signatures were then processed by the same scripts as before, which again successfully recovered the secret oil space.

7.1. Setup. We used a ChipWhisperer target board CW308-STM32F4 equipped with an ARM Cortex-M4 STM32F405RGT6 processor with 192KB RAM and running at a frequency of 7.8125 MHz. For our experiments, we inserted clock glitches during the initial computations with the goal of injecting a fault that generates a different target vector as the input of the linear system. We armed the board right before initiating the signature generation, as our attacks do not rely on precise triggering to be effective. In fact, a fairly broad time range can be utilized to inject a fault, hence practically showing the power and effectiveness of our attacks. In practice, fault injection parameters were identified through trial and error. Specifically, we estimated the fault injection offset from the trigger by simple instructions counting using **gdb** and iterated over several fault injection parameter values until usable faulty signatures were generated.

7.2. Attacks on MAYO. We performed our experiments before the latest update of the MAYO specification for round 2 of the NIST process. We note that the parameter changes between round 1 and round 2 shown in Table 2 and Table 3 do not affect our attack. In our experiments, we implemented the first fault attack described in Section 5.2. In this attack, we target a pseudo-random number generation. This process was not changed between round 1 and round 2.

For our experiments, we utilized the MAYO-1 reference implementation developed by Beullens et al. (available at <https://github.com/PQCMayo/MAYO-M4>, commit `fe46236`). The implementation was cross-compiled using the compiler **arm-none-eabi-gcc** version 13.2.1 and optimized with **-Oz**. We made a simple modification to the implementation to enable deterministic signature generation, as illustrated in Listing 1.

We then conducted a fault injection campaign with the following parameters: a width $8 \leq w \leq 10$, an offset $-15 \leq o \leq 15$, an external offset between $100 \leq o' \leq 500$, and a step $4 \leq s \leq 8$, with 3 repetitions. With this setup, we were able to obtain roughly 10 unique and exploitable faulty signatures per hour with a 20% fault injection success rate. For example, we were able to successfully inject faults using the parameters $(w, o, o', s) = (9, -15, 222, 0)$. An excerpt of the code that is – estimated by simple instruction counting using **gdb** – the location of a successful fault injection is shown in Listing 2. This is consistent with the attack described in 5.2. Using our scripts, it was possible to recover the secret key with just 2 signatures with 100% repeatability. These can either be 2 faulty signatures or a combination of a faulty signature and a correct one.

```

152 ...
153 // choose the randomizer
154 //if (randombytes(tmp + param_digest_bytes, param_salt_bytes)
    != MAYO_OK) {
155 //     ret = MAYO_ERR;
156 //     goto err;
157 //}
158
159 // deterministic MAYO
160 memset(tmp + param_digest_bytes, 0, param_salt_bytes);
161 ...
    
```

LISTING 1. Excerpt from the MAYO-1 signature generation function `mayo_sign` in file `MAYO-C/src /mayo.c`

```

...
188 // hashing to t
189 memcpy(tmp + param_digest_bytes, salt, param_salt_bytes);
190 ctrbyte = tmp + param_digest_bytes + param_salt_bytes +
    param_sk_seed_bytes;
191
192 shake256(tenc, param_m_bytes, tmp, param_digest_bytes +
    param_salt_bytes);
193 decode(tenc, t, param_m); // may not be necessary
...
    
```

LISTING 2. Excerpt from the MAYO-1 signature generation function `mayo_sign` in file `MAYO-C/src /mayo.c`

7.3. Attacks on PROV. In our experiments, we use the PROV1 reference implementation available at https://prov-sign.github.io/PROV_1-2_implem.zip¹. Unfortunately, the reference implementation does not fit STM32F405RGT6, because the expanded public key is 497513 bytes for PROV1 and the expanded secret key is 237469 bytes. Both are required during signing in the current implementation. Together with the compressed keys, this amounts to 816045 bytes, that does not fit within 192 KB RAM. Therefore, we made a few changes to adapt the code to our target controller. These changes do not affect the code sections where faults are injected, nor do they alter the implementation’s susceptibility to our attacks.

The implementation was cross-compiled using `arm-none-eabi-gcc` version 13.2.1 and optimized with `-Oz`. We then conducted a fault injection campaign with the following parameters: a width $2 \leq w \leq 10$, an offset $-5 \leq o \leq 5$, an external offset between $2600 \leq o' \leq 3000$, and a step $4 \leq s \leq 8$, with 3 repetitions. With this setup, we were able to obtain roughly 50 unique and exploitable faulty signatures per hour with a 15% fault injection success rate. For example, we were able to successfully inject faults using the parameters $(w, o, o', s) = (2, 2, 2704, 0)$. An excerpt of the code that is – estimated by simple instruction counting using `gdb` – the location of a successful fault injection for $o' = 2704$ is shown in Listing 3. This is the part of the code that needs to be targeted for the attack described in Section 6.2. Using our scripts, it was possible to recover the secret key with just 2 signatures with 100% repeatability. These can either be 2 faulty signatures or a combination of a faulty signature and a correct one.

¹Commit ID 4bd19a8f4bb205033743e2277c7aeef1

```

...
281     while(1) {
282
283         //new salt
284         HashSqueeze(&ctx_v, sig->salt, param->len_salt);
285
286         //new hash
287         HashInit(&ctx_msg, HASH_PREFIX_MESSAGE);
288         HashUpdate(&ctx_msg, expanded_skey->hashed_pkey,
289                  param->len_hashed_pkey);
289         HashUpdate(&ctx_msg, msg, len_msg);
290         HashUpdate(&ctx_msg, sig->salt, param->len_salt);
291         HashFinal(&ctx_msg);
292         HashSqueeze(&ctx_msg, target, param->m);
...

```

LISTING 3. Excerpt from the PROV1 signature generation
function `prov_extended_sign` in file `prov1/prov.c`

8. COUNTERMEASURES

Deterministic signature schemes based on different mathematical principles are known to be susceptible to fault attacks. Attacks against deterministic signature schemes based on elliptic curves are described, for example, in [25]. The paper sets out fault attacks against a deterministic variant of ECDSA [26] and against EdDSA [19]. In [17] attacks against deterministic variants of the lattice-base signature schemes Dilithium [5] and qTesla [1] are analyzed. Fault attacks against a deterministic variant of the lattice-based signature scheme Falcon are described in [6, 7]. On the other hand, implementations of non-deterministic signature schemes that require a reliable entropy source are sometimes flawed to the extent that these implementations are completely insecure [12]. Providing a good entropy source is a challenge when implementing cryptographic schemes. Therefore, some schemes – Dilithium [5], for example – use a ‘hedged’ approach and use both an external entropy source and the entropy contained in the message that is to be signed. On the other hand, there seem to be use cases for deterministic signature schemes [21].

Faulty signatures produced by a device under the attack in Section 4.1 are not valid signatures. Therefore, verifying the signature before outputting it would be a simple and effective countermeasure. The advantage of this countermeasure is that it would also detect other types of faults and may therefore protect against other attacks as well. The disadvantage of this countermeasure is that it incurs a noticeable performance penalty. From the figures in Table 3.1 in [10] for the performance of MAYO on AVX2, we see that signature verification requires about 20% to 40% of the clock cycles needed for signing. Hence, verifying every signature after generating it would create an overhead of 20% to 40% in the signing procedure.

The faulty signatures produced by our attack in Section 4.2, however, are valid signatures. Hence, verifying signatures before outputting them does not work as a countermeasure against our attacks. Because our attacks target pseudo-random generation and not mathematical operations, there does not seem to be a way to use the mathematical structure of the signing procedure to detect these faults.

Therefore, the most promising approach to designing countermeasures seems to be to address directly the place where the fault is injected, i.e., harden the pseudo-random generation against faults. For a pseudo-random generation based on SHAKE as in MAYO or PROV, fault-resistant implementations of Keccak as described in [15, 22] would be applicable.

9. CONCLUSION AND OUTLOOK

In this paper we have presented new fault attacks against signature generation in UOV-based signature schemes. We have shown how implementations of the deterministic variant of MAYO and PROV can be attacked. Key recovery requires only one correct and one faulty signature for the same message in the case of deterministic MAYO. In the case of PROV, one correct and two faulty signatures are required. With our attacks the attacker has a large time window for injecting a fault. Moreover, the precise type of the fault is not important. Single or multiple instruction skips or changed data can all lead to exploitable faults.

This paper shows once again that ‘hedged’ designs for signature schemes are preferable, i.e., designs where random numbers required for signature generation are obtained by a combination of real- and pseudo-random number generators. The security of these designs depends neither solely on true random numbers (which are prone to implementation errors and hardware failures) nor solely on pseudo-random numbers (which are often susceptible to fault injection attacks).

It would be interesting to transfer the results from this paper to other UOV-based signature schemes. VOX [23] is another example of a deterministic UOV-based signature scheme that may be susceptible to attacks based on the ideas in this paper. Our results are further evidence that more research into fault-resistant implementations of Keccak is important.

Acknowledgements This work was partly funded by the German Federal Ministry of Education and Research in the project Quoryptan through grant number 16KIS2033.

REFERENCES

- [1] Erdem Alkim et al. *The Lattice-Based Digital Signature Scheme qTESLA*. Cryptology ePrint Archive, Paper 2019/085. 2019. URL: <https://eprint.iacr.org/2019/085>.
- [2] Thomas Aulbach, Fabio Campos, and Juliane Krämer. *SoK: On the Physical Security of UOV-based Signature Schemes*. Cryptology ePrint Archive, Paper 2024/1818. 2024. URL: <https://eprint.iacr.org/2024/1818>.
- [3] Thomas Aulbach et al. “MAYo or MAY-not: Exploring Implementation Security of the Post-Quantum Signature Scheme MAYO Against Physical Attacks”. In: *2024 Workshop on Fault Detection and Tolerance in Cryptography (FDTC)*. 2024, pp. 28–33. DOI: 10.1109/FDTC64268.2024.00012.
- [4] Thomas Aulbach et al. *Recovering Rainbow’s Secret Key with a First-Order Fault Attack*. Cryptology ePrint Archive, Paper 2022/632. <https://eprint.iacr.org/2022/632>. 2022. URL: <https://eprint.iacr.org/2022/632>.
- [5] Shi Bai et al. *CRYSTALS-Dilithium Algorithm Specifications and Supporting Documentation (Version 3.1)*. Tech. rep. 2021.
- [6] Sven Bauer and Fabrizio De Santis. “A Differential Fault Attack Against Deterministic Falcon Signatures”. In: *Smart Card Research and Advanced Applications*. Ed. by Shivam Bhasin and Thomas Roche. Cham: Springer Nature Switzerland, 2024, pp. 43–61. ISBN: 978-3-031-54409-5.
- [7] Sven Bauer and Fabrizio De Santis. *A Differential Fault Attack against Deterministic Falcon Signatures*. Cryptology ePrint Archive, Paper 2023/422. 2023. URL: <https://eprint.iacr.org/2023/422>.

- [8] Ward Beullens. “Improved Cryptanalysis of UOV and Rainbow”. In: *Advances in Cryptology – EUROCRYPT 2021*. Ed. by Anne Canteaut and François-Xavier Standaert. Cham: Springer International Publishing, 2021, pp. 348–373. ISBN: 978-3-030-77870-5.
- [9] Ward Beullens. *MAYO: Practical Post-Quantum Signatures from Oil-and-Vinegar Maps*. Cryptology ePrint Archive, Paper 2021/1144. <https://eprint.iacr.org/2021/1144>. 2021. URL: <https://eprint.iacr.org/2021/1144>.
- [10] Ward Beullens et al. *MAYO*. Tech. rep. available at <https://csrc.nist.gov/Projects/pqc-dig-sig/round-1-additional-signatures>. National Institute of Standards and Technology, 2023.
- [11] Ward Beullens et al. *MAYO*. Tech. rep. available at <https://csrc.nist.gov/Projects/pqc-dig-sig/round-2-additional-signatures>. National Institute of Standards and Technology, 2025.
- [12] Joachim Breitner and Nadia Heninger. *Biased Nonce Sense: Lattice Attacks against Weak ECDSA Signatures in Cryptocurrencies*. Cryptology ePrint Archive, Paper 2019/023. 2019. URL: <https://eprint.iacr.org/2019/023>.
- [13] *FIPS PUB 202 SHA-3 Standard: Permutation-Based Hash and Extendable-Output Functions*. Federal Information Processing Standards Publication 202. National Institute of Standards and Technology (NIST), Aug. 2015. DOI: 10.6028/NIST.FIPS.202. URL: <https://doi.org/10.6028/NIST.FIPS.202>.
- [14] Hiroki Furue et al. “A New Fault Attack on UOV Multivariate Signature Scheme”. In: (2021). URL: https://ipsj.ixsq.nii.ac.jp/ej/?action=pages_view_main&active_action=repository_view_main_item_detail&item_id=214513&item_no=1&page_id=13&block_id=8.
- [15] Ivan Gavrilan et al. “Impeccable Keccak: Towards Fault Resilient SPHINCS+ Implementations”. In: *IACR Transactions on Cryptographic Hardware and Embedded Systems* 2024.2 (2024), pp. 154–189. DOI: 10.46586/tches.v2024.i2.154-189. URL: <https://tches.iacr.org/index.php/TCHES/article/view/11424>.
- [16] Louis Goubin et al. *PROV — PProvable unbalanced Oil and Vinegar*. Tech. rep. available at <https://csrc.nist.gov/Projects/pqc-dig-sig/round-1-additional-signatures>. National Institute of Standards and Technology, 2023.
- [17] Leon Groot Bruinderink and Peter Pessl. “Differential Fault Attacks on Deterministic Lattice Signatures”. In: *IACR Transactions on Cryptographic Hardware and Embedded Systems* 2018.3 (2018), pp. 21–43. DOI: 10.13154/tches.v2018.i3.21-43. URL: <https://tches.iacr.org/index.php/TCHES/article/view/7267>.
- [18] Sönke Jendral and Elena Dubrova. *MAYO Key Recovery by Fixing Vinegar Seeds*. Cryptology ePrint Archive, Paper 2024/1550. 2024. URL: <https://eprint.iacr.org/2024/1550>.
- [19] Simon Josefsson and Ilari Liusvaara. *RFC8032 – Edwards-Curve Digital Signature Algorithm (EdDSA)*. Tech. rep. 2017. URL: <https://datatracker.ietf.org/doc/html/rfc8032>.
- [20] Juliane Krämer and Mirjam Loiero. *Fault Attacks on UOV and Rainbow*. Cryptology ePrint Archive, Paper 2024/277. <https://eprint.iacr.org/2024/277>. 2024. URL: <https://eprint.iacr.org/2024/277>.
- [21] David Lazar and Chris Peikert. *Deterministic Falcon Signatures*. Tech. rep. 2021. URL: <https://github.com/algorand/falcon/blob/main/falcon-det.pdf>.
- [22] Hassen Mestiri, Imen Barraji, and Mohsen Machhout. “A High-Speed KECCAK Architecture Resistant to Fault Attacks”. In: *2020 32nd International*

- Conference on Microelectronics (ICM)*. 2020, pp. 1–4. DOI: 10.1109/ICM50269.2020.9331792.
- [23] Jacques Patarin et al. *VOX Specification v1.1*. 2024. URL: https://vox-sign.com/files/vox_1-1_spec.pdf.
- [24] Pierre Pébereau. *One vector to rule them all: Key recovery from one vector in UOV schemes*. Cryptology ePrint Archive, Paper 2023/1131. 2023. URL: <https://eprint.iacr.org/2023/1131>.
- [25] Damian Poddebniak et al. “Attacking Deterministic Signature Schemes Using Fault Attacks”. In: *2018 IEEE European Symposium on Security and Privacy (EuroS&P)*. 2018, pp. 338–352. DOI: 10.1109/EuroSP.2018.00031.
- [26] Thomas Pornin. *RFC 6979 – Deterministic Usage of the Digital Signature Algorithm (DSA) and Elliptic Curve Digital Signature Algorithm (ECDSA)*. Tech. rep. Internet Engineering Task Force, 2013. URL: <https://datatracker.ietf.org/doc/html/rfc6979>.
- [27] Kyung-Ah Shim and Namhun Koo. “Algebraic Fault Analysis of UOV and Rainbow With the Leakage of Random Vinegar Values”. In: *IEEE Transactions on Information Forensics and Security* 15 (2020), pp. 2429–2439. DOI: 10.1109/TIFS.2020.2969555.